



Wallet Control

Auditoría técnica, hallazgos de seguridad y plan a versión 1.0
— Actualización de avance de Fase 1 —

Preparado con Claude Code · claude-workspace
2 de julio de 2026 · actualizado 4 de julio de 2026

Wallet Control es una app de finanzas personales con asesor de IA integrado ("Finando"), construida en Expo/React Native. Este documento resume la auditoría técnica original del 2 de julio de 2026 y el re-chequeo del 4 de julio contra el código real del proyecto, para confirmar qué puntos de la Fase 1 (bloqueantes) siguen pendientes.

Estado de la Fase 1 al 4 de julio de 2026: 3 de 6 puntos resueltos — la fase NO está completa.

Siguen pendientes: el proxy/backend para la IA, la configuración de EAS (eas.json) y la limpieza de assets (ícono y splash). Sin estos tres puntos no se puede generar un build instalable ni activar la IA real de forma segura.

Hallazgos clave

BLOQUEANTE

Seguridad — antes de compartir un build con IA real

- PENDIENTE**

1. API key de Anthropic expuesta en el bundle — `lib/claude.ts:91,103` . La llamada a la API se sigue haciendo directo desde el cliente con la key leída de `EXPO_PUBLIC_ANTHROPIC_API_KEY` . Cualquier variable `EXPO_PUBLIC_*` queda incrustada en el bundle JS de producción — extraíble descompilando el APK. Verificado el 4 de julio: sin cambios respecto al hallazgo original.
- RESUELTO**

2. Contraseñas en texto plano — el campo `password` ya no existe en `lib/storage.ts` ni en ningún otro archivo del proyecto (verificado por búsqueda en todo el código). El riesgo fue eliminado de raíz en vez de mitigado.
- RESUELTO**

3. Modelo desactualizado — `lib/claude.ts` ya usa `claude-sonnet-5` en vez de `claude-sonnet-4-6` .

BLOQUEANTE

Build y publicación — sigue sin poder generarse un APK/IPA

- RESUELTO**

4. ios.bundleIdentifier y android.package — ambos presentes en `app.json` (`com.walletcontrol.app`).
- PENDIENTE**

5. No existe eas.json — sigue sin correrse `eas build:configure` ; no hay perfiles de build configurados.
- RESUELTO**

6. expo-notifications como plugin — ya está declarado en `plugins` dentro de `app.json` .
- PENDIENTE**

7. Assets rotos — verificado visualmente: `icon.png` sigue con las guías de diseño (círculos y ejes) sin limpiar, y `splash-icon.png` sigue siendo la plantilla vacía, sin logo.

Plan a versión 1.0

Fase 1 — Cerrar bloqueantes

- ☐ Mover la llamada a Finando IA a un backend/proxy (o mantener el modo demo como modo público mientras no exista proxy)
- ☒ Eliminar o asegurar el campo password — *hecho: campo eliminado por completo*
- ☒ Agregar bundleIdentifier / package y el plugin de expo-notifications en app.json — *hecho*

- ☐ Correr eas build:configure para generar eas.json
- ☐ Re-exportar íconos y splash sin las guías de diseño
- ☒ Migrar a claude-sonnet-5 cuando se active la IA real — *hecho*

Los tres puntos pendientes son los que bloquean tanto compartir la IA real (proxy) como generar el build instalable (EAS + assets). Recomendado resolverlos antes de intentar `eas build --profile preview`.

Fase 2 — Pulido de producto (no re-verificada en este chequeo)

#	HALLAZGO	ARCHIVO
8	Promesa de "Excel/CSV próximamente" que no existe ni está en desarrollo	app/ayuda.tsx
9	Markdown del chat de Finando se muestra literal (asteriscos visibles)	components/ChatBubble.tsx
10	Componente stub vacío, código muerto	components/QuincenaCard.tsx
11	Componente de presupuesto completo pero nunca montado en ninguna pantalla	components/BudgetProgressBar.tsx
12	Falta catch en el login — error no capturado	app/onboarding.tsx
13	No avisa si se deniega el permiso de notificaciones	components/CardFormModal.tsx
14	Versión desincronizada: perfil muestra v0.6.0, package.json dice 1.0.0	app/(tabs)/perfil.tsx
15	Sin tests automatizados (aceptable para el tamaño actual)	—
16	Sin accessibilityLabel/Role en botones de solo-ícono	toda la app

Fase 3 — Roadmap de features (priorizado desde NOTAS/ideas wallet-control.txt)

Después de publicar 1.0, con feedback real primero: notificaciones de presupuesto, exportar Excel/CSV, búsqueda de gastos, comparativa mes a mes. Para v2.0 o con tracción: modo pareja/familia, categorización automática por IA, integración bancaria.

Cómo empezar a compartirla y conseguir audiencia

- 1

Build compatible sin fricción. Con la Fase 1 realmente cerrada (los 3 puntos pendientes), correr `eas build -platform android --profile preview` genera un APK instalable directo, compatible por link a un grupo cerrado de confianza (5-10 personas).
- 2

Beta cerrada (2-3 semanas). Recoger feedback específico (¿usan Finando IA?, ¿entienden el modo anónimo?, ¿qué confunde del onboarding?) antes de exponerse a audiencia más grande. En paralelo, subir a Google Play Internal Testing (gratis).
- 3

Fachada pública en paralelo. Landing de una sola página con la skill PAKI: qué hace la app, 2-3 screenshots reales, botón de descarga o lista de espera.
- 4

Lanzamiento público. Play Store (cuenta de desarrollador, \$25 único pago) + Product Hunt + comunidades de finanzas personales en español (subreddits, grupos de Telegram/Facebook, comunidades locales de Colombia).
- 5

Ciclo de audiencia continuo. Usar el historial de ACTUALIZACIONES.txt como base para publicar novedades en redes cada vez que se agregue algo del roadmap — mantiene interés y da razones para volver a abrir la app.

Verificación

- **Fase 1 completa cuando:** el build preview de EAS corre sin errores y genera un APK instalable, sin la API key visible en el bundle. (*Aún no — faltan proxy, eas.json y assets*)
- **Fase 2:** probar el flujo completo — onboarding → registrar gasto por chat → ver categoría en Resumen → exportar PDF — en el APK generado, no solo en Expo Go.
- **Fase 4:** confirmar que al menos 3-5 personas fuera del creador instalaron y usaron la app desde el APK compartido antes de subir a Play Store.

Documento generado automáticamente a partir de la auditoría de código de wallet-control · claude-workspace
Re-verificado contra el código real el 4 de julio de 2026